

ARGUS // Unified Cyber Defense Platform

TECHNICAL ARCHITECTURE, PHASE SPECIFICATIONS & DIRECTORY MAPPING

1. Executive System Overview

ARGUS is an integrated, high-assurance cybersecurity intelligence platform designed to evaluate IPsec VPN cryptographic configurations, detect security misconfigurations, and perform real-time machine learning inference to forecast network attack vectors before exploitation occurs. By unifying VPN configuration inspection with time-series packet anomaly detection, ARGUS provides security operations centers (SOCs) with unified posture scores and automated countermeasures.

2. Operational Architecture & End-to-End Flow

Component Layer	Technology Stack	Primary Functionality
Phase 1: VPN Testbed	Docker, StrongSwan, tcpdump	Deploys IPsec VPN gateways & captures pcap traffic
Phase 2: Packet Parser	Rust (pcap-parser, nom)	Parses IKEv2/ESP headers & evaluates cipher security
Phase 3: Feature Engineering	Python 3, pandas, numpy	Extracts 4D feature vectors [src_port, dst_port, proto, len]
Phase 4: ML Models	scikit-learn, joblib	Trains DecisionTree classifier & saves serialized model
Phase 5: Backend API	FastAPI, uvicorn, Pydantic	REST API for real-time inference & security reports
Phase 6: Frontend SOC	Next.js 16, Tailwind, Recharts	Tactical black & white UI dashboard with live graphs
Phase 7: Orchestration	Docker Compose, Shell Scripts	One-click containerized deployment & automated tests

3. Detailed Phase Breakdown — Phases 1 & 2

Phase 1 — VPN Testbed & Capture (phase1-vpn-testbed/):

Configures a containerized strongSwan IPsec gateway (IKEv2 with AES-256-GCM / SHA-256) inside Docker. Traffic generation scripts simulate benign VPN connections alongside adversarial traffic vectors (DDoS surges, port recon, and exploit probes). Raw packets are captured into PCAP files stored in phase1-vpn-testbed/capture/pcap_store/.

Phase 2 — High-Performance IPsec Parser (phase2-ipsec-parser/):

Written in Rust for microsecond-level throughput. Key files include src/main.rs (CLI entrypoint), src/parser.rs (zero-copy pcap binary header decoder), and src/report.rs (security score evaluator). Generates structured JSON security reports detailing encryption strength, DH group size, Perfect Forward Secrecy (PFS) state, and security scores (0-100).

4. Detailed Phase Breakdown — Phases 3, 4 & 5

Phase 3 — Feature Engineering (phase3-feature-engineering/):

Converts parsed packet streams into normalized feature matrices ready for machine learning consumption. Main modules:

- src/data_labeler.py: Reads PCAP/CSV captures, extracts 4 key network attributes (src_port, dst_port, protocol, length), and assigns ground-truth threat labels (0 = Normal, 1 = Suspicious).
- Outputs clean dataset files into data/synthetic_vpn_traffic.csv for training.

Phase 4 — Machine Learning Model Development (phase4-ml-model/):

Trains and evaluates predictive classification algorithms to detect threat anomalies in network flows.

- src/train.py: Ingests the feature dataset, performs StandardScaler normalization, trains a Decision Tree classifier, and outputs evaluation metrics (accuracy, precision, recall, confusion matrix).
- Exported Model Artifacts: Serialized binaries decision_tree.pkl and scaler.pkl stored in models/ directory.

Phase 5 — REST API Backend (phase5-backend/):

FastAPI microservice providing real-time REST inference and health monitoring.

- app/main.py: Initializes FastAPI app, CORS middleware, and global exception handlers.
- app/routes.py: Defines endpoints /api/v1/health, /api/v1/predict, and /api/v1/analyze.
- app/models.py: Pydantic request/response schemas with namespace protection.
- app/utlis.py: Robust ModelLoader class for loading decision_tree.pkl and executing inferencing.

5. Backend REST API Endpoint Specification

Endpoint Route	HTTP Method	Payload / Parameters	Description & Output
/api/v1/health	GET	None	Returns system health status & model load confirmation status
/api/v1/predict	POST	JSON: {src_port, dst_port, protocol, length}	Returns prediction class (0/1), attack type string & confidence score
/api/v1/analyze	POST	JSON: Feature key-value dictionary	Full telemetry report, risk level, confidence meter & recommendations

6. Project Workspace Directory Map

- unified-cyber-platform/
 - phase1-vpn-testbed/ # Docker strongSwan IPsec VPN container & PCAP captures
 - phase2-ipsec-parser/ # Rust binary packet parser & security scoring engine
 - phase3-feature-engineering/ # Python feature extraction & dataset labeling
 - phase4-ml-model/ # Decision Tree training & serialized model generation
 - phase5-backend/ # FastAPI server (main.py, routes.py, models.py, utlis.py)
 - phase6-frontend/ # Next.js SOC Dashboard (app/page.tsx, components, Recharts)
 - deploy.sh & demo.sh # One-click deployment & automated verification scripts
 - README.md # Project documentation & operational quick-start guide

7. Detailed Phase Breakdown — Phases 6 & 7

Phase 6 — Frontend SOC Command Dashboard (phase6-frontend/):

High-performance Next.js Turbopack web application crafted with a tactical black & white cybersecurity aesthetic, monospace font typography (JetBrains Mono), emerald green matrix status lights, and colorful Recharts analytics visualization.

- app/page.tsx: Central command center dashboard integrating form inputs, security score gauge, and live charts.
- app/components/Navbar.tsx: Header with live UTC telemetry clock, DecisionTree status pill, and active defense badge.
- app/components/PredictionForm.tsx: Tactical input panel featuring 1-click test vector presets (Normal VPN, DDoS Flood, Port Recon, VPN Exploit).
- app/components/SecurityScore.tsx: Animated SVG circular security score ring gauge & latency stats.
- app/components/ResultsDisplay.tsx: Real-time inference report card displaying risk level, confidence percentage bar, and security recommendations.
- app/components/ThreatAnalytics.tsx: Area trend stream & bar charts visualizing packet throughput and attack vector distributions.

Phase 7 — Container Orchestration & Automated Testing (Root Scripts):

- deploy.sh: One-click orchestration script that compiles Rust parser, installs backend dependencies, builds Next.js frontend, and launches uvicorn + dev servers.
- demo.sh: Automated validation script executing sample API requests and outputting end-to-end verification metrics.

8. Machine Learning Model Benchmark

Metric Name	Value / Benchmark	Operational Significance
Primary Classifier	Decision Tree (scikit-learn)	Provides deterministic, reproducible rule splits with zero latency
Inference Latency	< 3.0 milliseconds	Enables real-time inline packet stream classification without lag
Feature Vector	4 Parameters (src, dst, proto, len)	Lightweight payload extraction compatible with eBPF and pcap
Target Classes	Normal (0) / Suspicious (1)	Immediate binary alert triggering for SOC automated response

9. Quick Operating & Verification Guide

1. Launch Backend API Server (Port 8000):

cd phase5-backend && python3 -m uvicorn app.main:app --host 0.0.0.0 --port 8000

2. Launch Frontend SOC Dashboard (Port 3000):

cd phase6-frontend && npm run dev

3. Access Dashboard & Execute Analysis:

Navigate to `http://localhost:3000`. Select one of the quick test vector presets (e.g. *Normal VPN* or *DDoS Flood*) and click **EVALUATE_TRAFFIC_VECTOR**. The AI inference report and security gauge will update instantaneously.